

OS Practical 2520030331 Arnav chitikena

Program 1

```
GNU nano 8.7.1 vaddress.c *
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

OUTPUT

```
arnav@LAPTOP-BN8992KC:~/OS$ nano vaddress.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc vaddress.c -o vaddress
arnav@LAPTOP-BN8992KC:~/OS$ ./vaddress
Value of x = 10
Virtual address of x = 0x7ffc23f0a4c4
arnav@LAPTOP-BN8992KC:~/OS$ |
```

Program 2

```
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address : %p\n", (void *)main);
    printf("Data address : %p\n", (void *)&global);
    printf("BSS address : %p\n", (void *)&bss_variable);
    printf("Heap address : %p\n", (void *)heap);
    printf("Stack address : %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

OUTPUT

```

arnav@LAPTOP-BN8992KC:~/OS$ nano memory.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc memory.c -o memory
arnav@LAPTOP-BN8992KC:~/OS$ ./memory
Code address : 0x5a0c4e0521a9
Data address : 0x5a0c4e055010
BSS address : 0x5a0c4e055018
Heap address : 0x5a0c69af8010
Stack address : 0x7ffd58ae684c

```

PROGRAM 3

```

#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}

```

OUTPUT

```

arnav@LAPTOP-BN8992KC:~/OS$ nano memory.c
arnav@LAPTOP-BN8992KC:~/OS$ nano pagetable.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc pagetable.c -o pagetable
arnav@LAPTOP-BN8992KC:~/OS$ ./pagetable
Enter page number (0-3): 2
Page is already in memory.
Physical frame = 3
arnav@LAPTOP-BN8992KC:~/OS$ |

```

PROGRAM 4

```

#include <stdio.h>

int main() {

    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number      = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}

```

OUTPUT

```

arnav@LAPTOP-BN8992KC:~/OS$ nano translation.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc translation.c -o translation
arnav@LAPTOP-BN8992KC:~/OS$ ./translation
Virtual Address = 250
Page Number      = 2
Offset           = 50
Frame Number      = 1
Physical Address = 150

```

PROGRAM 5

```

GNU nano 8.7.1 DMA_malloc.c
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);
}

```

OUTPUT

```
arnav@LAPTOP-BN8992KC:~/OS$ nano DMA_malloc.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc DMA_malloc.c -o DMA_malloc
arnav@LAPTOP-BN8992KC:~/OS$ ./DMA_malloc
10 20 30 40 50
```

PROGRAM 6

```
GNU nano 8.7.1 null.c
#include <stdio.h>

int main() {
    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

OUTPUT

```
arnav@LAPTOP-BN8992KC:~/OS$ gcc null.c -o null
arnav@LAPTOP-BN8992KC:~/OS$ ./null
Before accessing memory
Segmentation fault ./null
```